

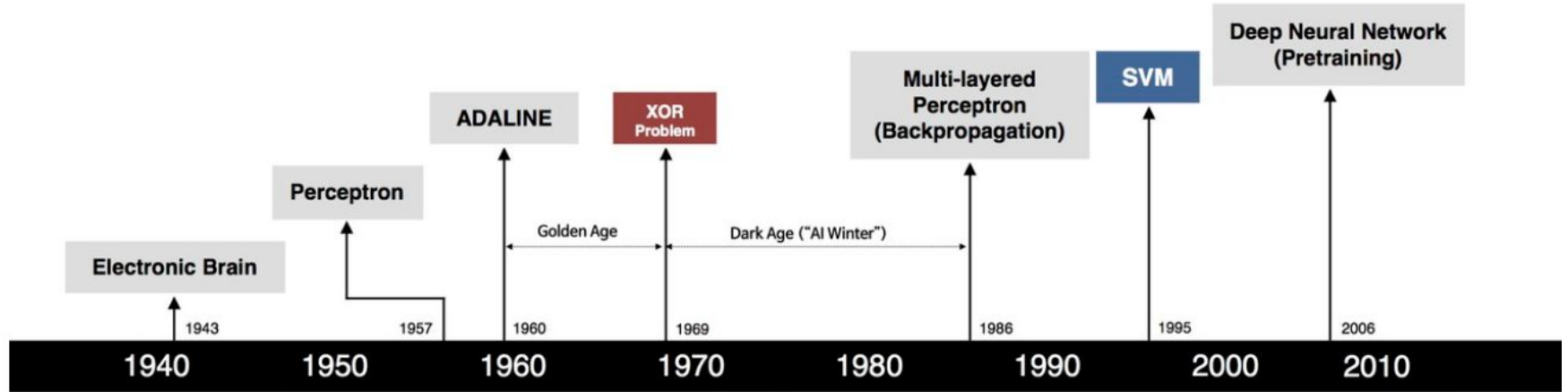
Unidad 4 -Aprendizaje profundo

# Diplomatura en Inteligencia Artificial

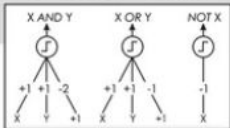
## TUIA - 2025

Prof: Lic Alan Geary - email: [alan.geary.b@gmail.com](mailto:alan.geary.b@gmail.com) 

# Clase 3- Aprendizaje profundo (DL)



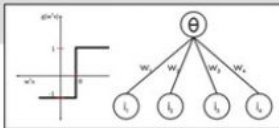
S. McCulloch - W. Pitts



- Adjustable Weights
- Weights are not Learned



F. Rosenblatt



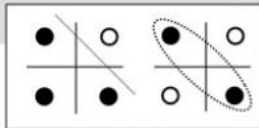
- Learnable Weights and Threshold



B. Widrow - M. Hoff



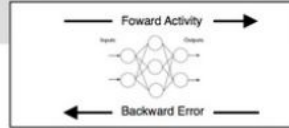
M. Minsky - S. Papert



- XOR Problem



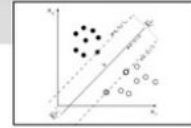
D. Rumelhart - G. Hinton - R. Williams



- Solution to nonlinearly separable problems
- Big computation, local optima and overfitting



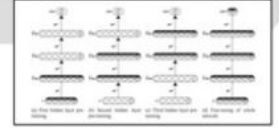
V. Vapnik - C. Cortes



- Limitations of learning prior knowledge
- Kernel function: Human Intervention



G. Hinton - S. Ruslan



- Hierarchical feature Learning

# Clase 3- Aprendizaje profundo (DL)

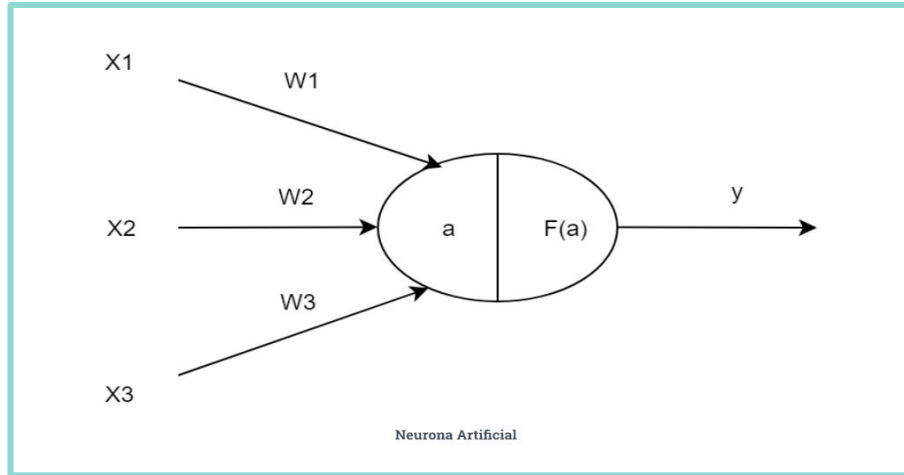
## 3.1 Recordando el perceptrón



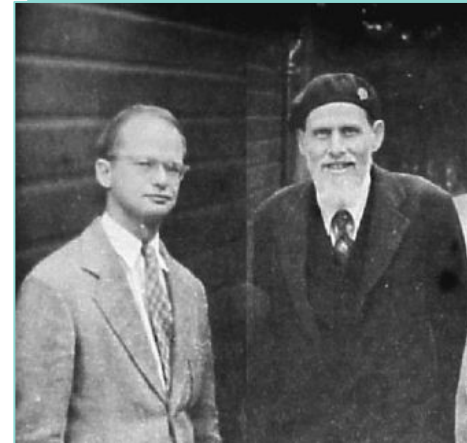
# 3.1 - Perceptrón

En 1943 **McCulloch** y **Pitts** propusieron un modelo de neurona artificial denominado LTU ( Linear Threshold Unit)

*“Un cálculo lógico de las ideas inmanentes a la actividad nerviosa”*



McCulloch y Pitts



## 3.1 - Perceptrón

Parten de varias hipótesis, 3 de ellas son las que más representan el método:

1. **La actividad de la neurona es un proceso "todo-o-nada".**
2. **Deberán excitarse un cierto número fijo de sinapsis dentro del período de adición latente para excitar a la neurona en cualquier momento, y este número es independiente de la actividad previa y la posición de la neurona.**
3. **La actividad de cualquier sinapsis inhibidora impide completamente la excitación de la neurona en ese momento.**

## 3.1 - Perceptrón

### 1. La actividad de la neurona es un proceso "todo-o-nada".

Esto supone que la neurona es una función binaria que transforma un conjunto de valores booleanos en otro valor booleano.

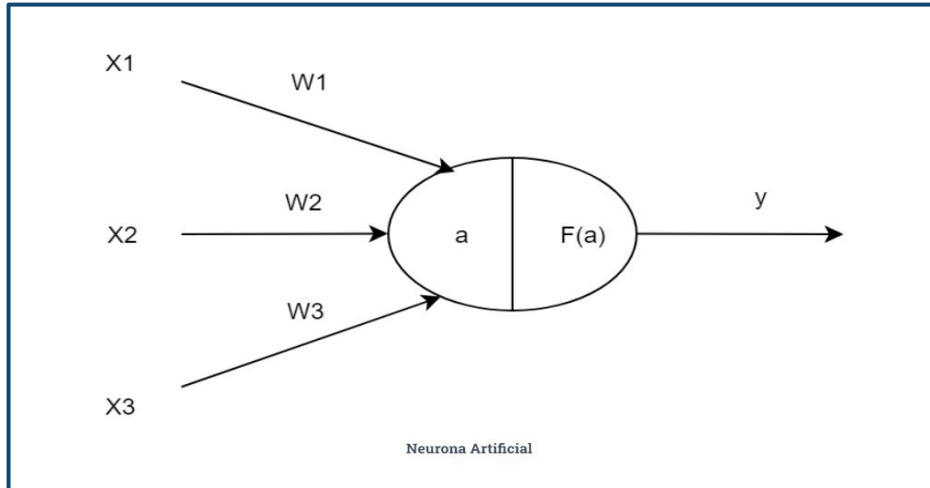
### 2. Deberán excitarse un cierto número fijo de sinapsis

Tendrá asociado un umbral fijo que determinará el grado de excitabilidad de la neurona. Cuando el número de señales excitadoras iguale o supere este umbral, la neurona se excitará y generará una señal de valor 1 que se transmitirá a la siguiente.

## 3.1 - Perceptrón

### 3. Sinapsis inhibitoras.

Si alguna señal de entrada es inhibidora siempre la salida (Y) será 0.

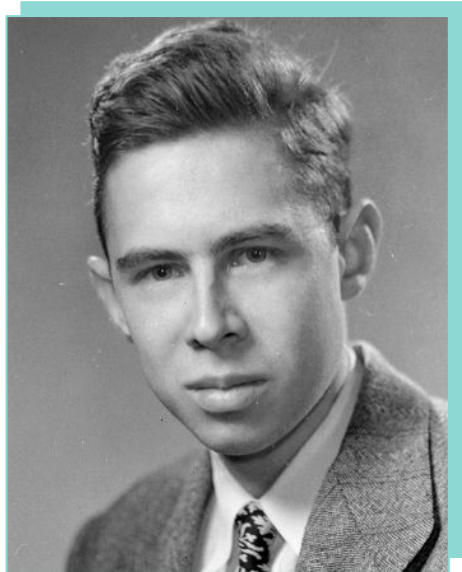


$X_i$  son (0,1)

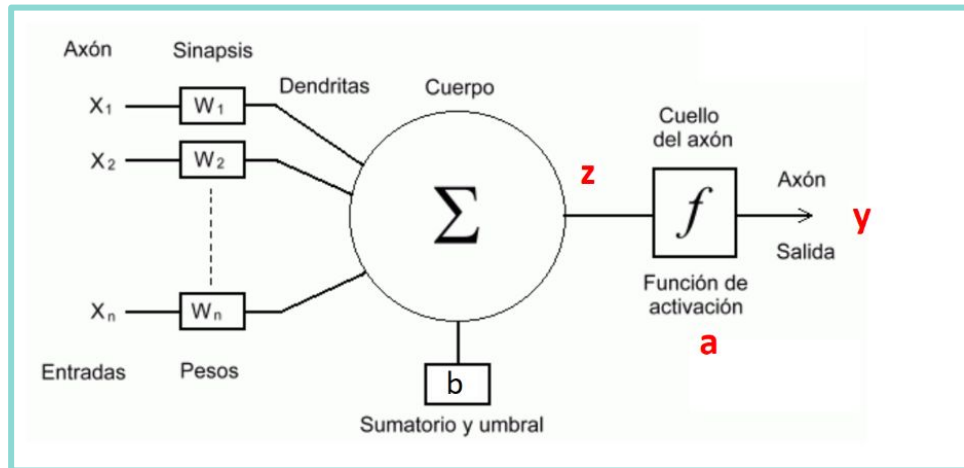
$Y$  es (0,1)

# 3.1 - Perceptrón

En 1958, **Frank Rosenblatt**, neurobiólogo de Cornell, comenzó a trabajar en el **perceptrón**.



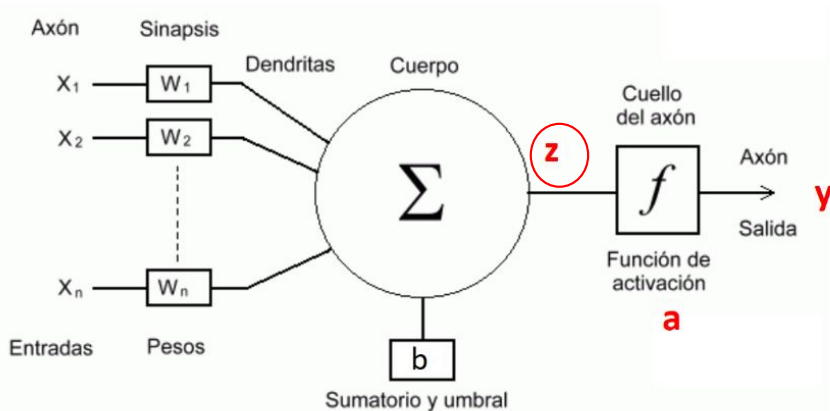
***“THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN”***





## 3.1 - Perceptrón

Un **perceptrón** es un **modelo de clasificación** que consta de un conjunto de ponderaciones o puntuaciones, una para cada característica, y un umbral. El perceptrón multiplica cada ponderación por su puntuación correspondiente y las suma, obteniendo una puntuación. Si esta puntuación es mayor o igual que el umbral, el perceptrón devuelve un «sí», un «verdadero», o el valor 1. Si es menor, devuelve un «no», un «falso», o el valor 0.



$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

## 3.1 - Perceptrón

| Característica              | McCulloch & Pitts (1943)    | Perceptrón de Rosenblatt (1958)   |
|-----------------------------|-----------------------------|-----------------------------------|
| Tipo de entrada             | Binaria: $x_i \in \{0, 1\}$ | Continua: $x_i \in \mathbb{R}$    |
| Pesos                       | Fijos, usualmente positivos | Reales y ajustables (aprendizaje) |
| Umbral (threshold)          | Número entero fijo          | Representado como un sesgo $b$    |
| Función de activación       | Escalón binario             | Escalón binario                   |
| Salida                      | Binaria: $y \in \{0, 1\}$   | Binaria: $y \in \{0, 1\}$         |
| Aprendizaje                 | No permite aprendizaje      | Sí, mediante regla de aprendizaje |
| Propósito original          | Modelar funciones lógicas   | Clasificación supervisada         |
| Capacidad de representación | Lógica proposicional        | Problemas linealmente separables  |

# 3.1 - Perceptrón

## Procedimiento:

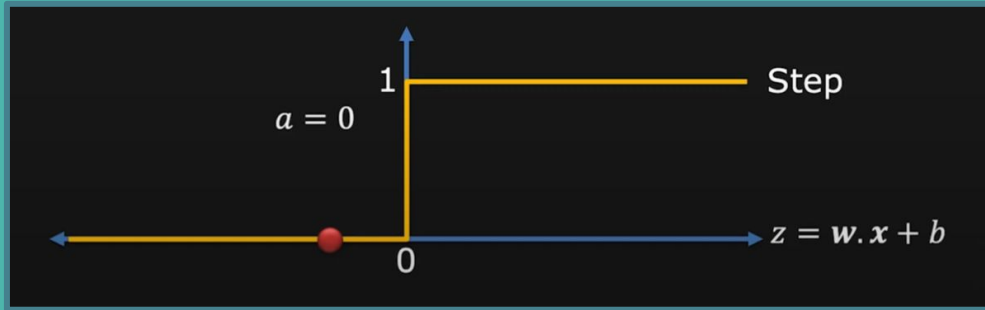
- Comienza con una línea aleatoria. En otras palabras, comienza con valores aleatorios para la puntuación de cada palabra y el sesgo.
- Repita el siguiente procedimiento n veces:
  - Elige un punto al azar.
  - Si el punto está bien clasificado, no hagas nada; si está mal clasificado, acerca la línea un poco más al punto.

¡Disfruta de tu línea bien ajustada!

EJEMPLO

## 3.1 - Perceptrón, la función escalón

- La salida cambia de 0 a 1 en un único salto.
- No existe una graduación que indique “qué tan lejos” está un ejemplo del umbral.

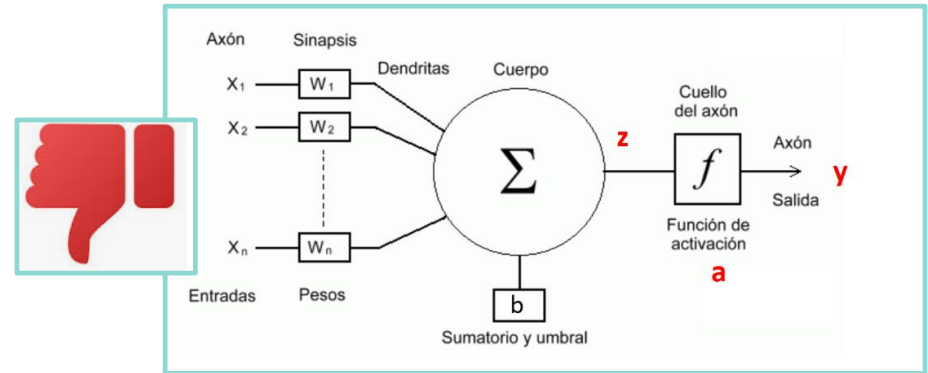


$$\text{where, } f(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ 1 & \text{if } z > 0 \end{cases}$$

Geométricamente, eso equivale a dibujar una línea recta que separa los puntos “rojos” de los “celestes”.

## 3.1 - Perceptrón, la crítica.

En 1969 **Minsky** y **Papert** demuestran que el perceptrón simple no puede resolver problemas no lineales

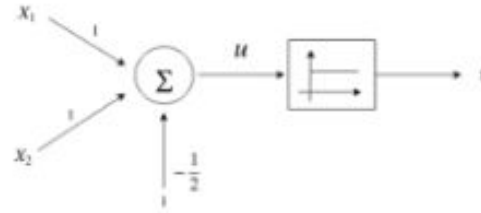
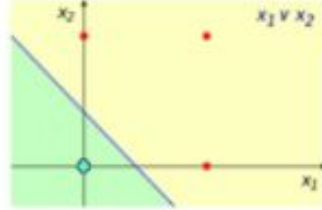


[A Review of "Perceptrons: An Introduction to Computational Geometry"](#)

# Posibilidades y alcances de un Perceptrón

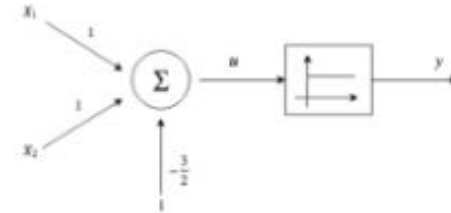
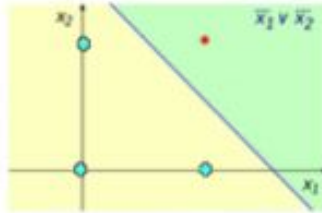
OR

| ENTRADAS |   | SALIDAS |
|----------|---|---------|
| X        | Y | X or Y  |
| 0        | 0 | 0       |
| 1        | 0 | 1       |
| 0        | 1 | 1       |
| 1        | 1 | 1       |



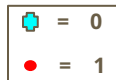
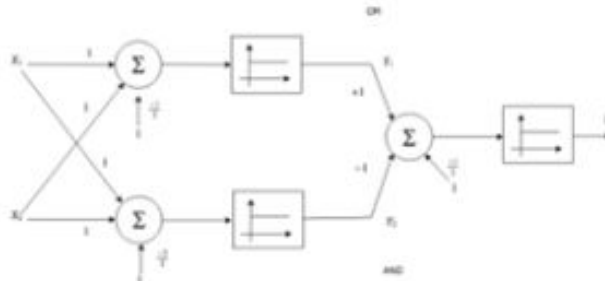
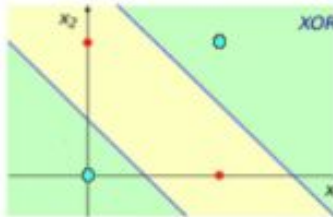
AND

| ENTRADAS |   | SALIDAS |
|----------|---|---------|
| X        | Y | X and Y |
| 0        | 0 | 0       |
| 1        | 0 | 0       |
| 0        | 1 | 0       |
| 1        | 1 | 1       |



XOR

| ENTRADAS |   | SALIDAS |
|----------|---|---------|
| X        | Y | X xor Y |
| 0        | 0 | 0       |
| 1        | 0 | 1       |
| 0        | 1 | 1       |
| 1        | 1 | 0       |



EJEMPLO

# Winter is coming



A partir de las críticas en el libro “Perceptrons” la inteligencia se fundamentó en las investigaciones y desarrollo simbólicos. Pero en la década del 70’ fue evidente que este paradigma entraba en crisis dando paso a lo que llamamos el Invierno de la IA.

---

Los 80'



# Diferenciabilidad y funciones de activación continuas

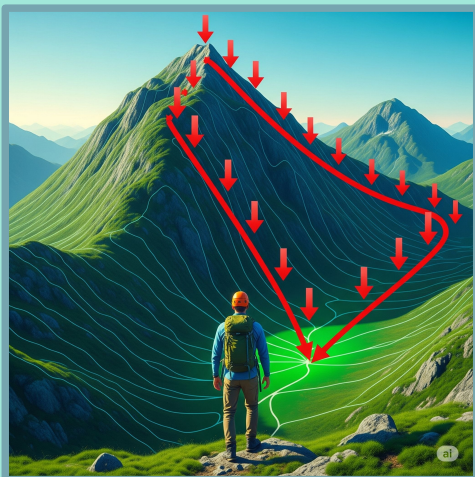


## La función de activación del perceptrón

Para usar **descenso de gradiente** **necesitas derivadas**. El escalón es plano (derivada = 0) en todos lados y tiene un salto infinito justo en el umbral. Resultado:

- **Dentro de cada región plana (en el mundo paramétrico)** la derivada es cero  $\rightarrow$  el algoritmo no sabe hacia dónde mover los pesos.
- **En el salto (la parte vertical de la función escalón)** la derivada ni siquiera está definida  $\rightarrow$  no se puede usar en un cálculo sistemático.

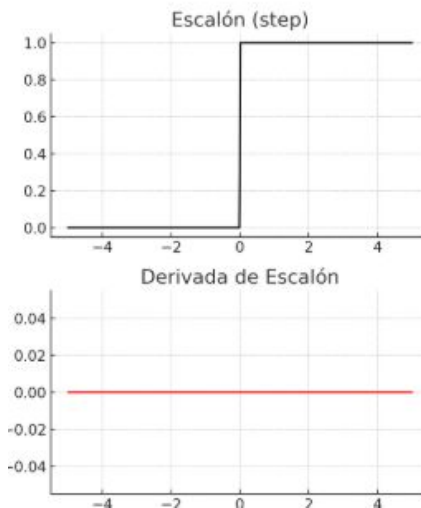
# Diferenciabilidad y funciones de activación continuas



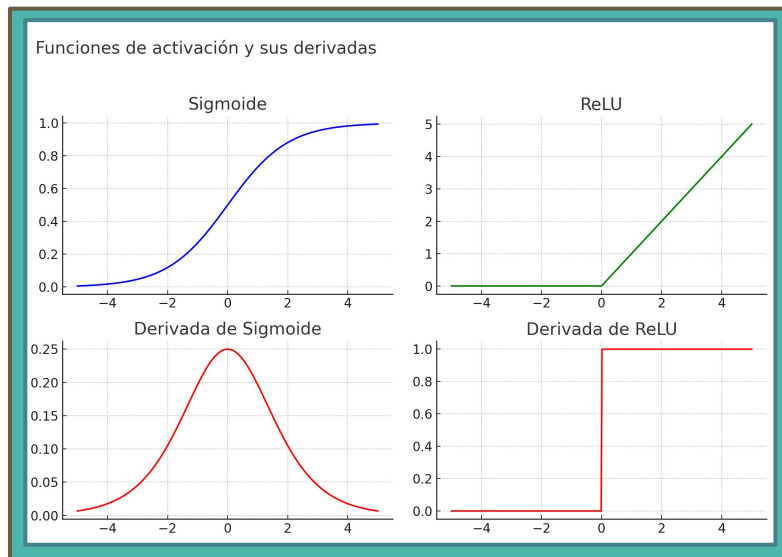
Imagina que querés encontrar el camino para bajar una montaña **con los ojos vendados**, usando solo el tacto para sentir si el terreno sube o baja.



Con la función escalón  
Estás en una meseta plana (no sabés si subir o bajar)  
**Resultado:** no podés aprender

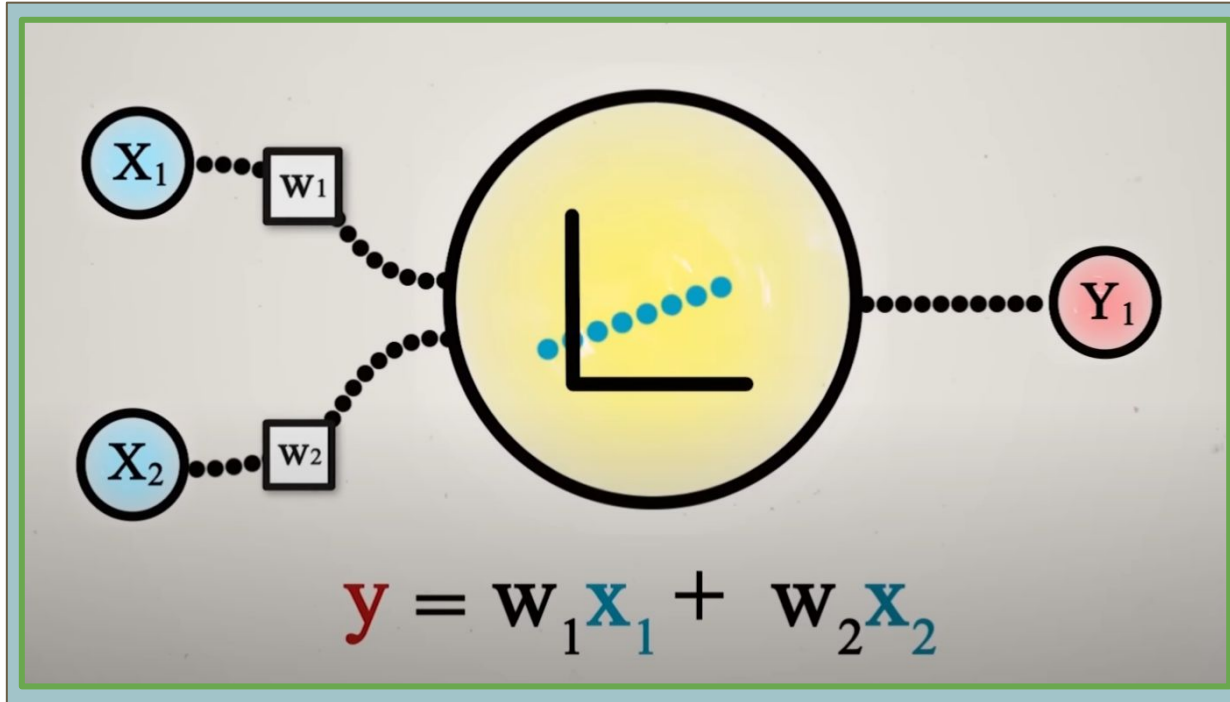


# Diferenciabilidad y funciones de activación continuas

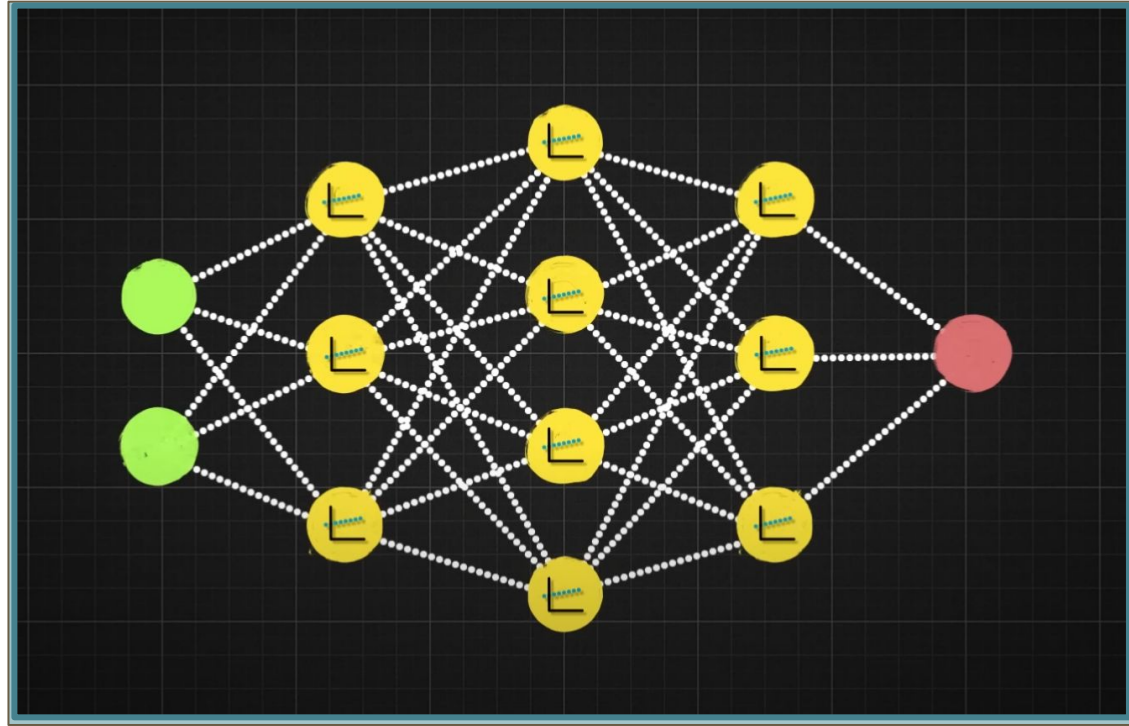


Al ser continuas y diferenciables (al menos a trozos), convirtieron la salida de cada neurona en una **función derivable respecto a sus pesos (parámetros)**. Ese simple cambio abrió la puerta a un **cálculo exacto de gradientes**, algo que el perceptrón clásico no permitía y que resultó esencial para todo lo que vino después.

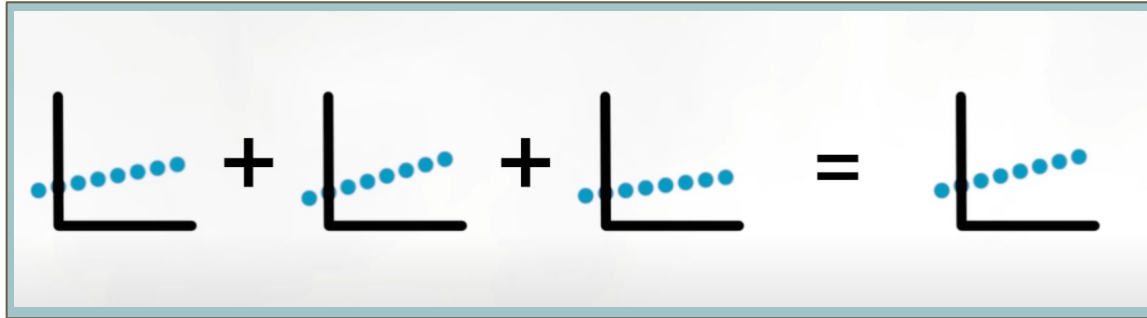
## 3.2 - Redes neuronales



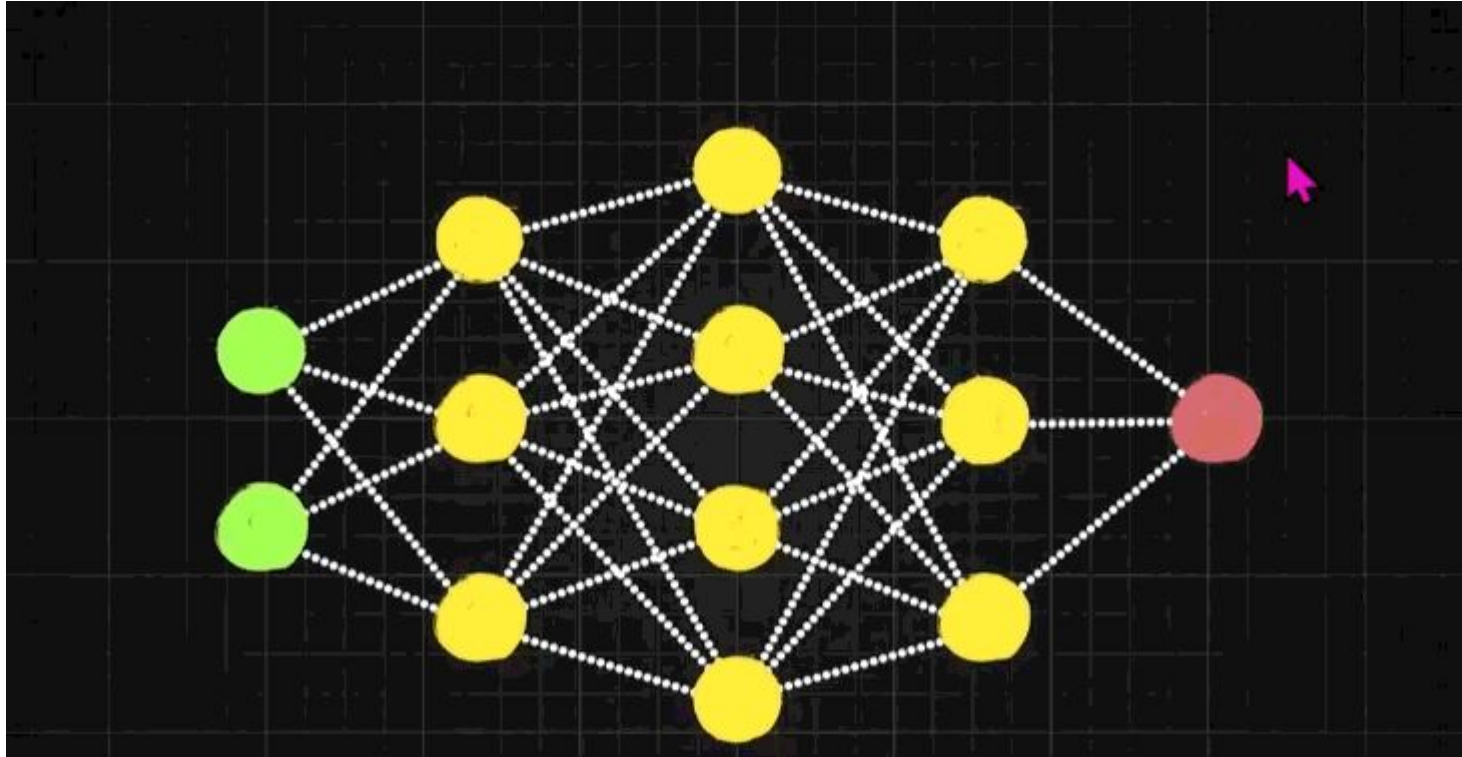
## 3.2 - Redes neuronales



## 3.2 - Redes neuronales

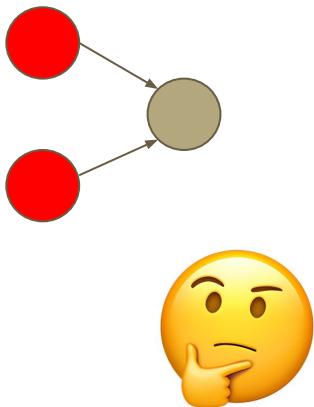


## 3.2 - Redes neuronales

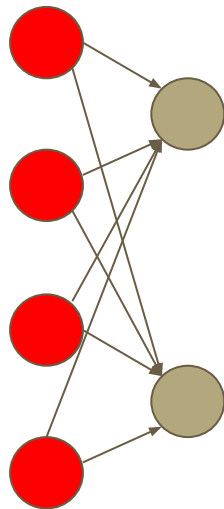


# ¿Por qué “profundo”(deep)?

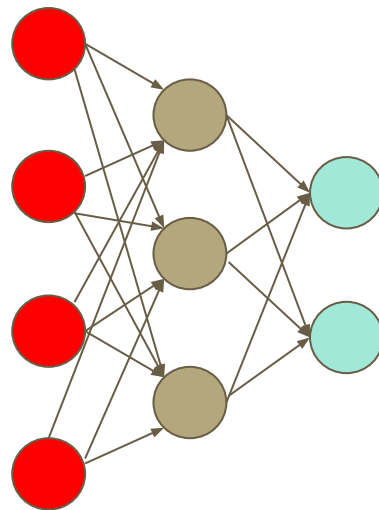
Perceptrón



red neuronal monocapa

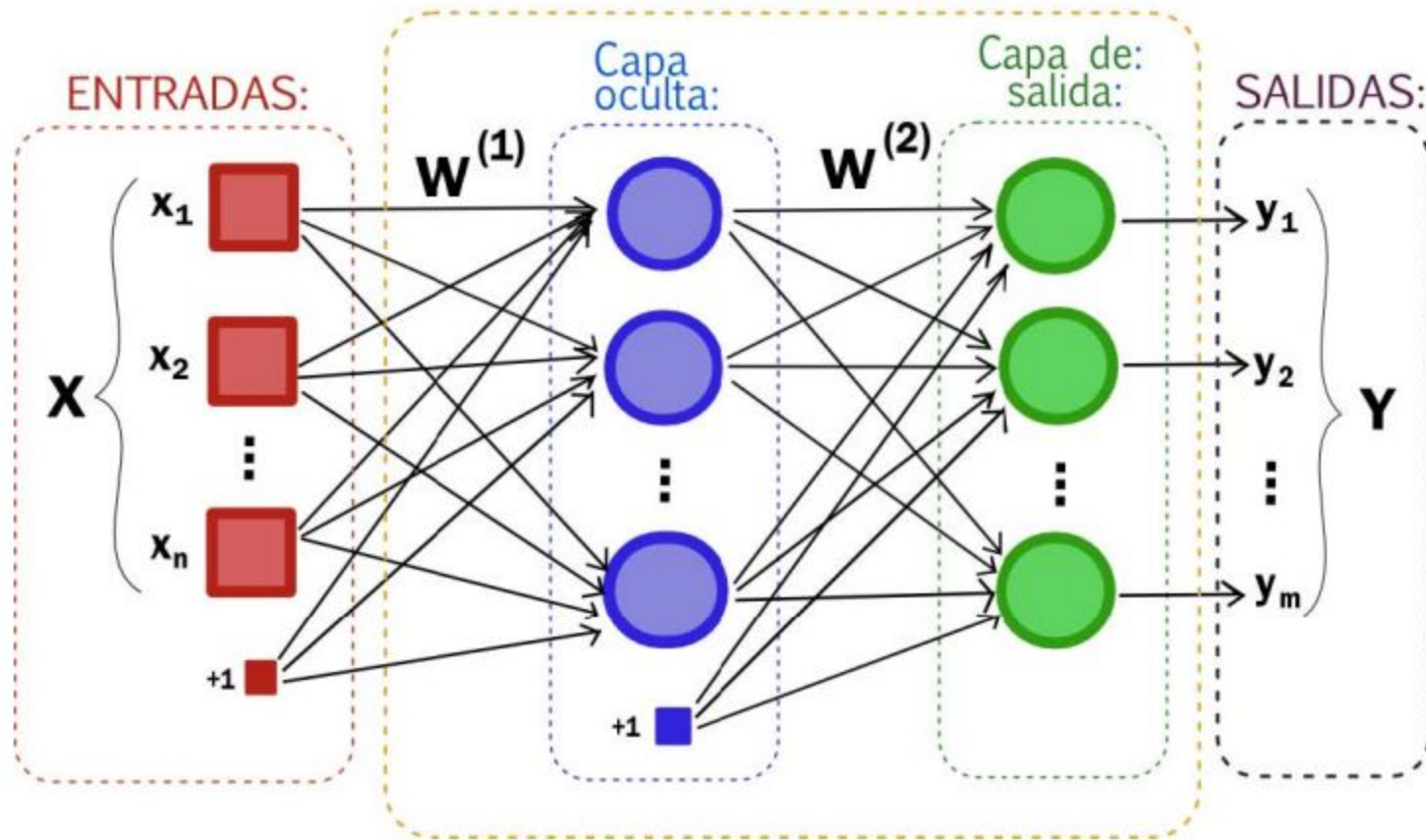


Red neuronal multicapa

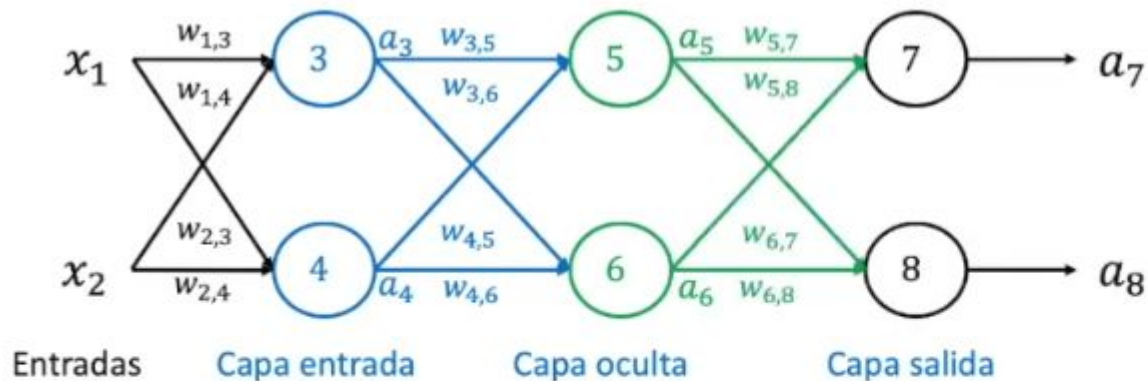




## RED NEURONAL:



# Feed forward, conexión hacia adelante



1. Unidades 3 y 4:

- $a_3 = g(x_1 w_{1,3} + x_2 w_{2,3})$ .
- $a_4 = g(x_1 w_{1,4} + x_2 w_{2,4})$ .

2. Unidades 5 y 6:

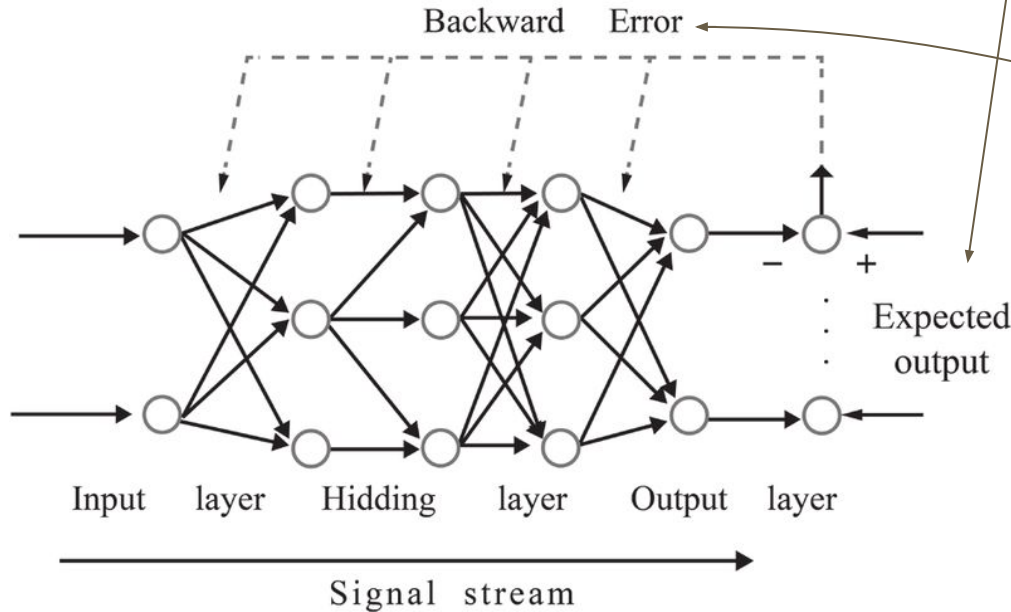
- $a_5 = g(a_3 w_{3,5} + a_4 w_{4,5})$ .
- $a_6 = g(a_3 w_{3,6} + a_4 w_{4,6})$ .

3. Unidades 7 y 8:

- $a_7 = g(x_1 w_{1,3} + x_2 w_{2,3})$ .
- $a_8 = g(x_1 w_{1,4} + x_2 w_{2,4})$ .

<https://playground.tensorflow.org/>

# Los avances teóricos “ Retropropagación”

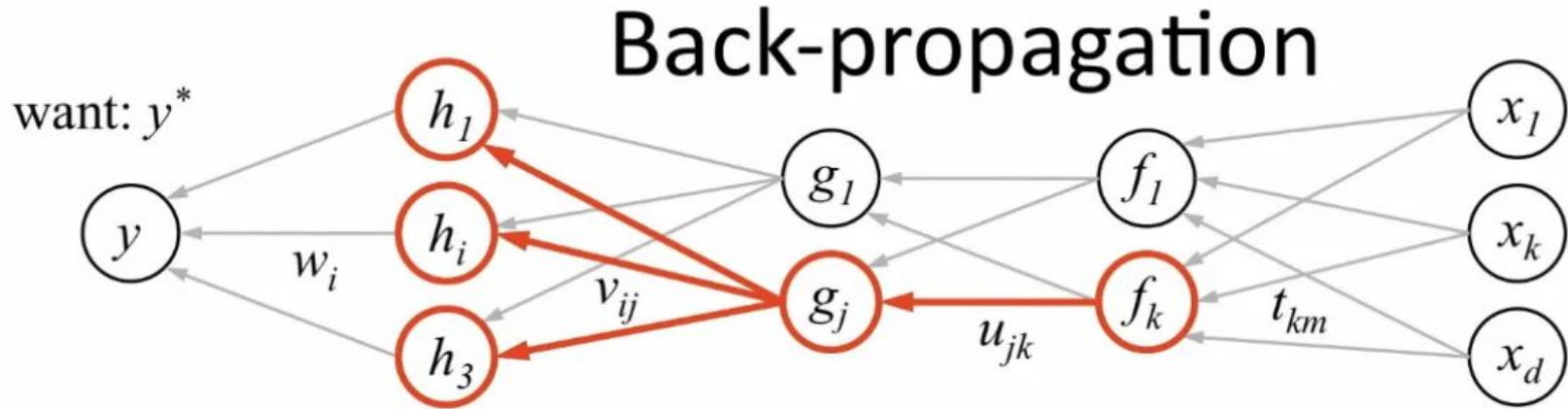


En la salida de la red compara el resultado estimado con el verdadero. Obtenemos un error/costo/diferencia.

Luego, usando esa diferencia/error busca “**la culpa**” de esa diferencia entre todos los pesos que participaron en la estimación. Cada peso va a tener un grado de culpabilidad el cual podemos obtener con el gradiente.

# Los avances teóricos “ Retropropagación”

Luego, ajusta los pesos que fueron encontrados culpables con el grado correspondiente y así se trata de reducir el error.



El **aprendizaje** en las redes neuronales consiste en modificar los pesos gradualmente para reducir el error en la salida y hacerlo cercano a 0 para todos los ejemplos de entrenamiento.

# Redes neuronales importantes

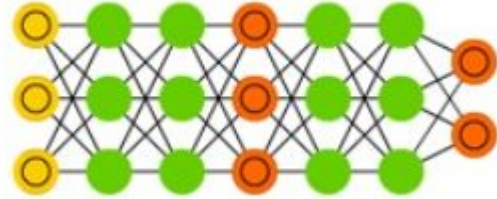
Deep Feed Forward (DFF)



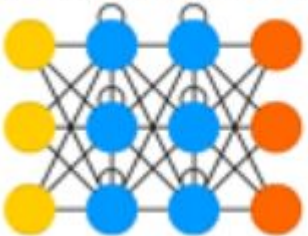
Long / Short Term Memory (LSTM)



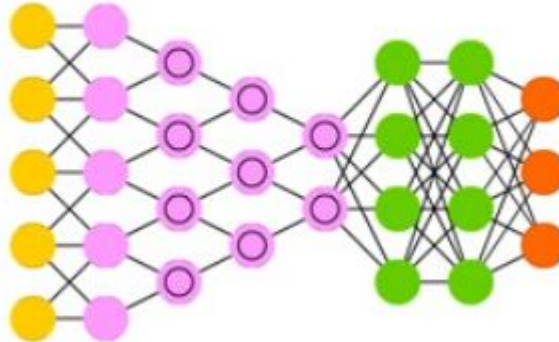
Generative Adversarial Network (GAN)



Recurrent Neural Network (RNN)



Deep Convolutional Network (DCN)



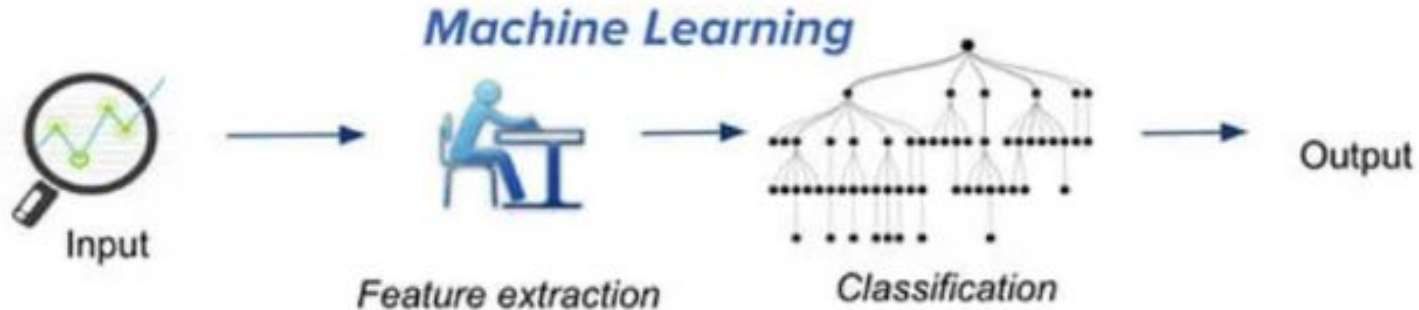
Auto Encoder (AE)



# CARACTERÍSTICAS (features)

# Diferencia con el machine learning tradicional

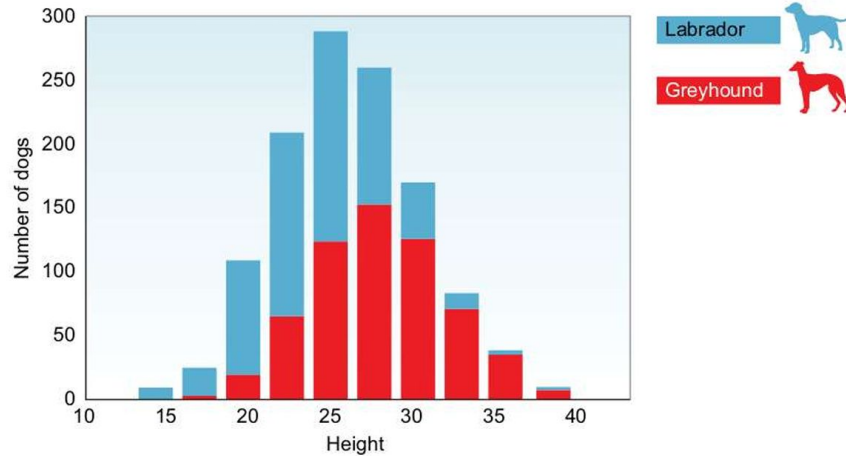
En el **Machine Learning** tradicional, la selección y extracción de características (features) es un proceso donde interviene un individuo a partir de los datos de entrada.



Lo importante es que las distribuciones de los valores observados de la característica presenten variabilidad entre los tipos de individuos. Esa diferencia es la información que toma el modelo.

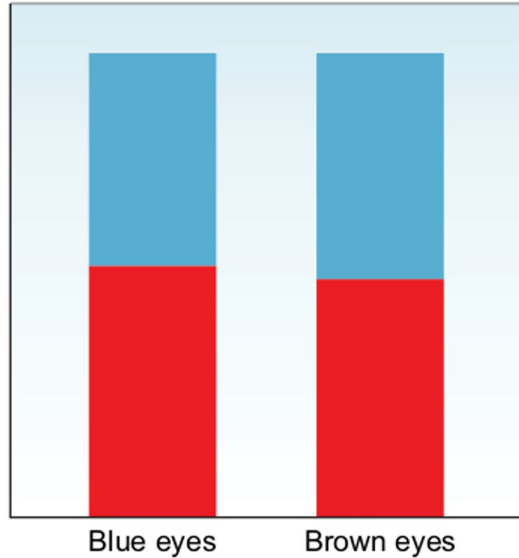


# ¿Cómo seleccionar una característica?



¿ Qué hace que la característica Altura (height) sea útil ?

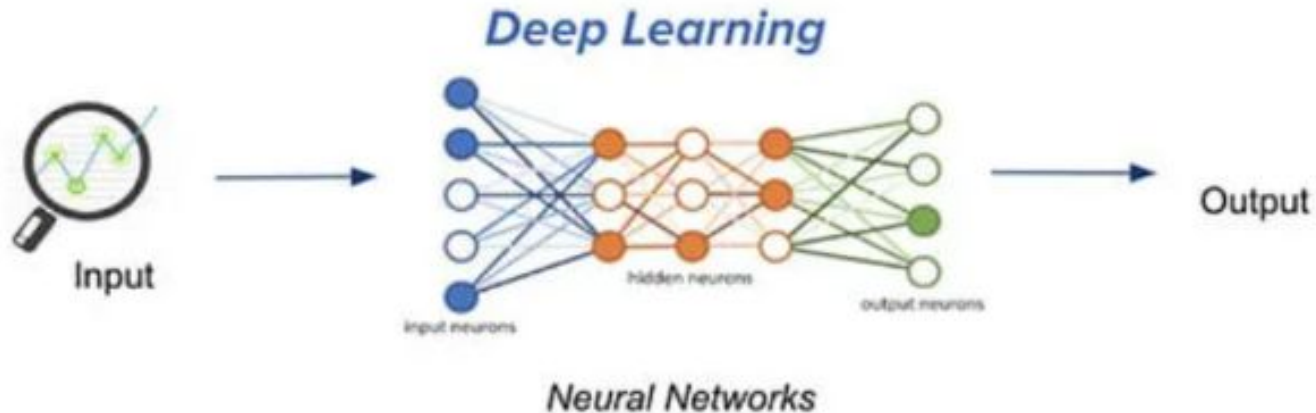
# ¿Cómo seleccionar una característica?



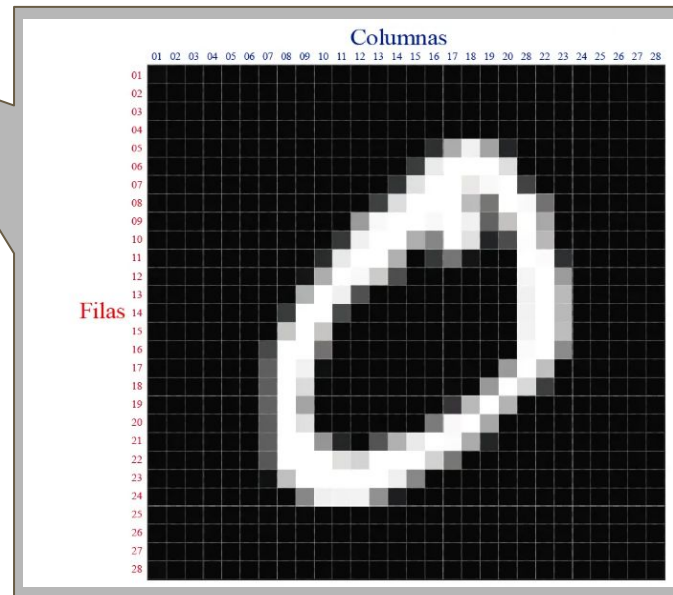
¿ Qué hace que la característica color de ojos no sea útil ?

# Diferencia con el deep learning

En **Deep Learning**, la red extrae los features automáticamente y aprende su importancia en el resultado final aplicando pesos a las conexiones. Las redes neuronales pueden ser pensadas como extractores de features + clasificadores que son entrenables de fin a fin.



# El dataset MNIST



El dataset MNIST es el considerado "Hello World" de la visión artificial. Contiene un conjunto de entrenamiento de 60.000 imágenes de dígitos manuscritos (de 0 a 9), y otro conjunto de pruebas con 10.000 muestras adicionales.

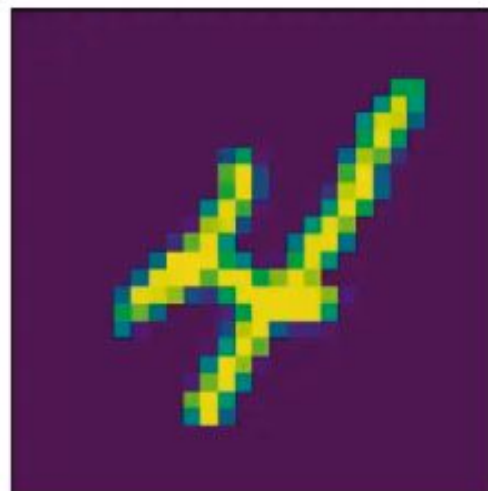
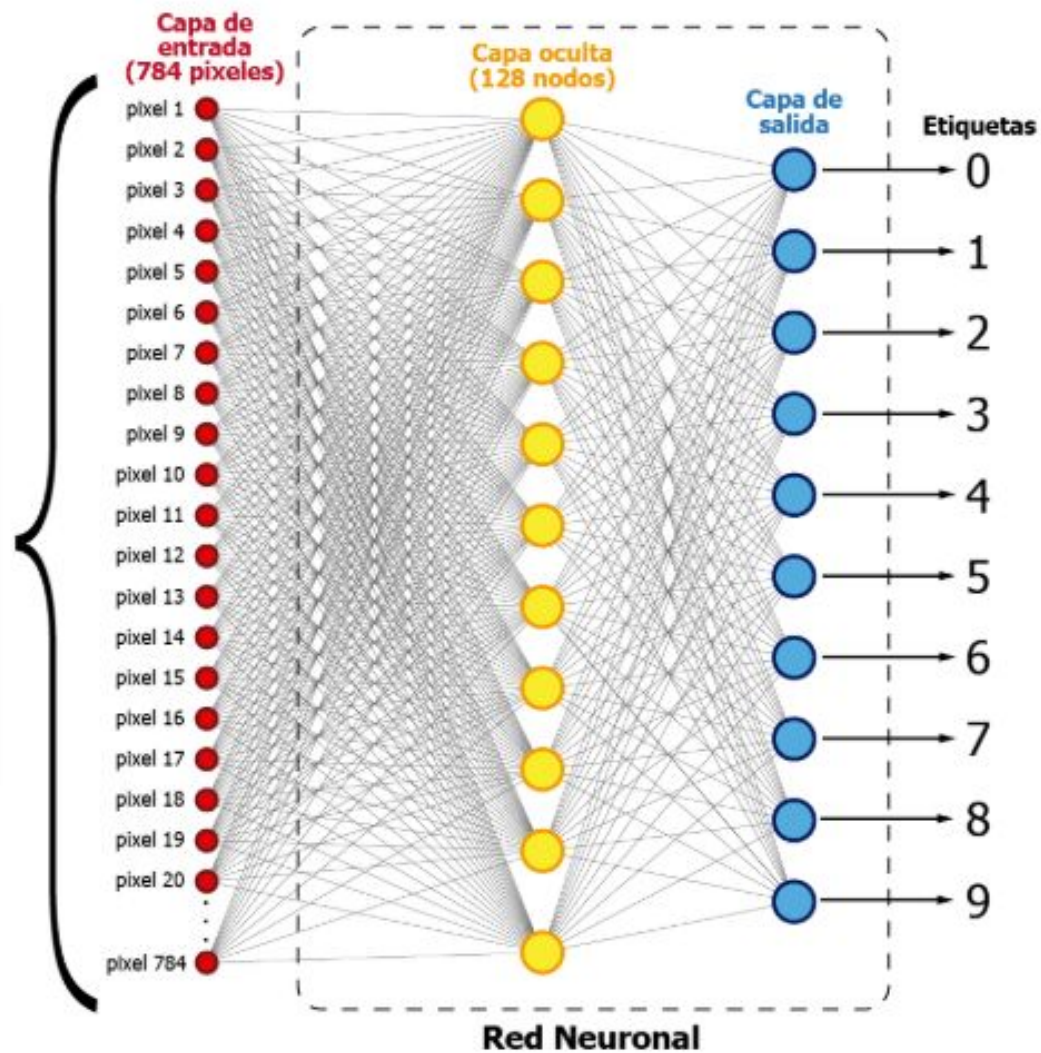
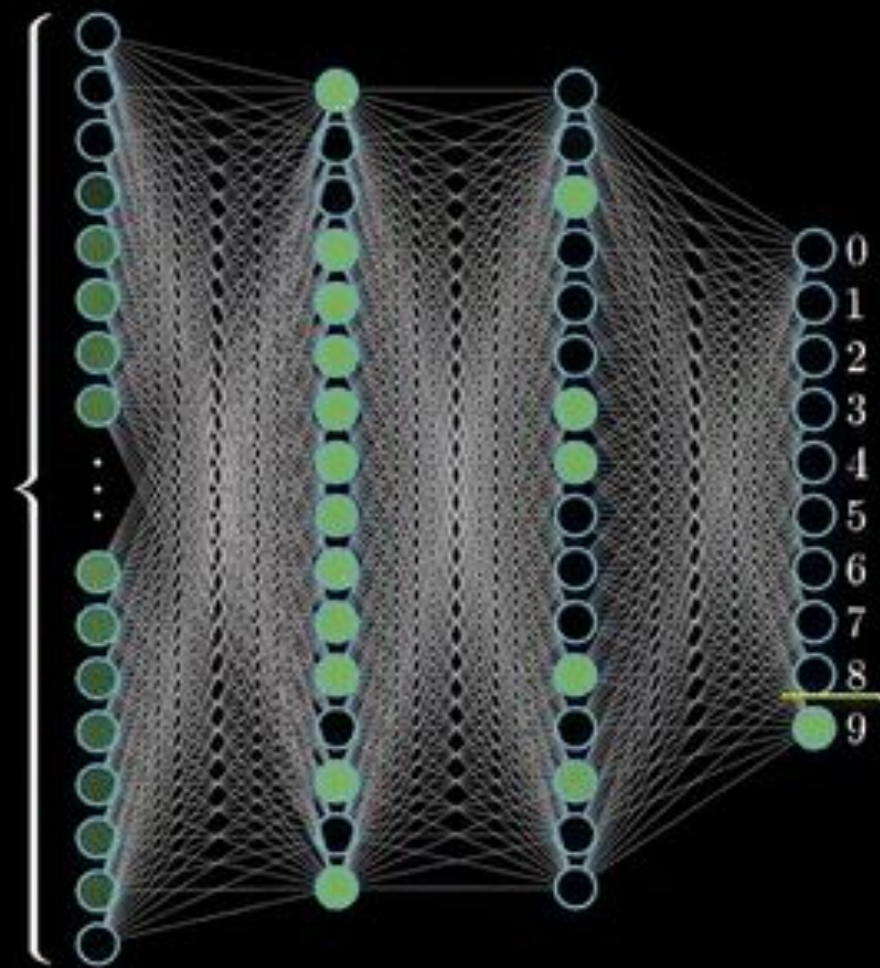


Imagen que se desea clasificar





784



Distintas Redes  
distintos usos



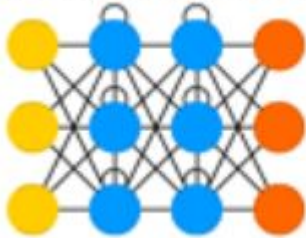
# RNN (Redes Neuronales Recurrentes)

Deep Feed Forward (DFF)



Cada capa permite profundizar en la **complejidad**. Y permite la jerarquía de información. Pero cada entrada es independiente no toma en cuenta el pasado.

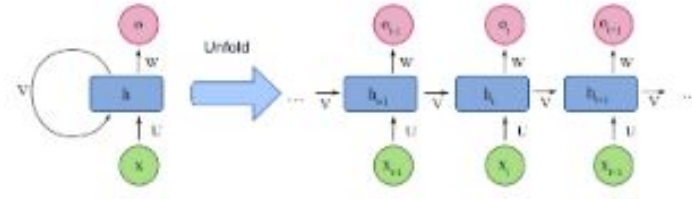
Recurrent Neural Network (RNN)



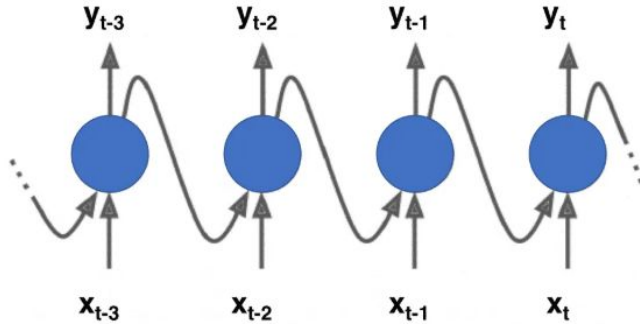
Las **rnn** pueden modelar secuencias y tipos de información con autocorrelación. Tomando en cuenta la incidencia de momentos anteriores.



# RNN (Redes Neuronales Recurrentes)



# El problema de la memoria



1 - Exploding Gradients  
2 - Vanishing Gradients

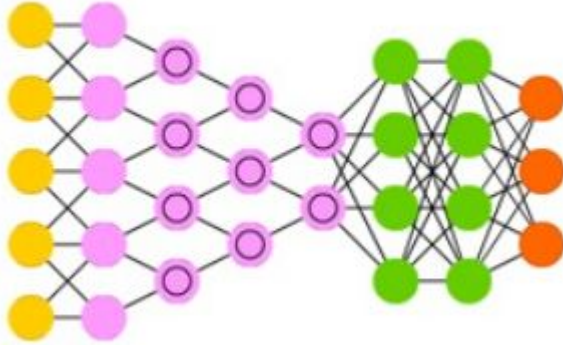
Long / Short Term Memory (LSTM)



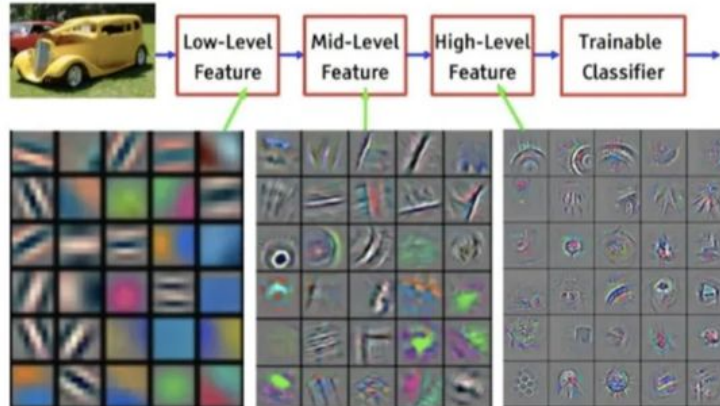
Las **LSTM** vienen a resolver el problema con células de memoria.

# Redes convolucionales

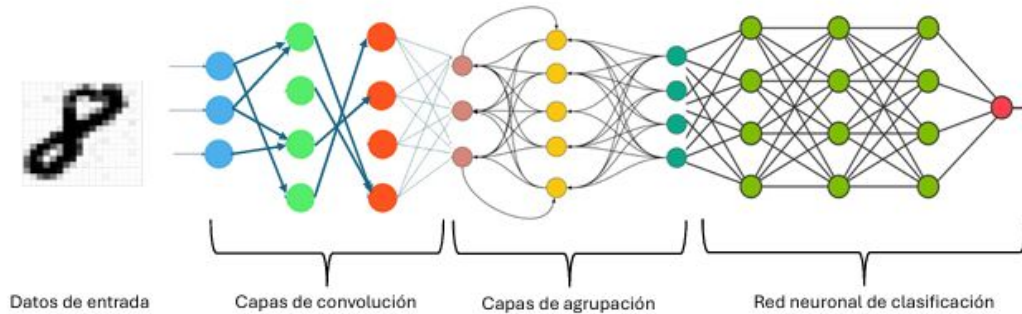
Deep Convolutional Network (DCN)



La red convolucional está pensada para sacar partido de la estructura 2D de una imagen.



# Redes para imágenes

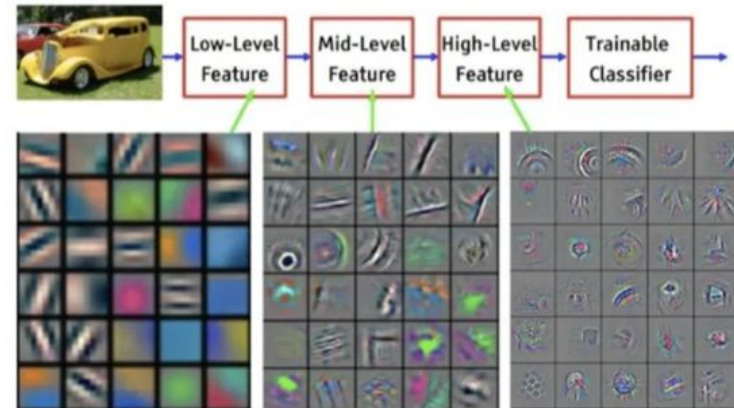
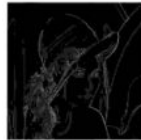


Input Image \* Filter (Kernel) = Output Image



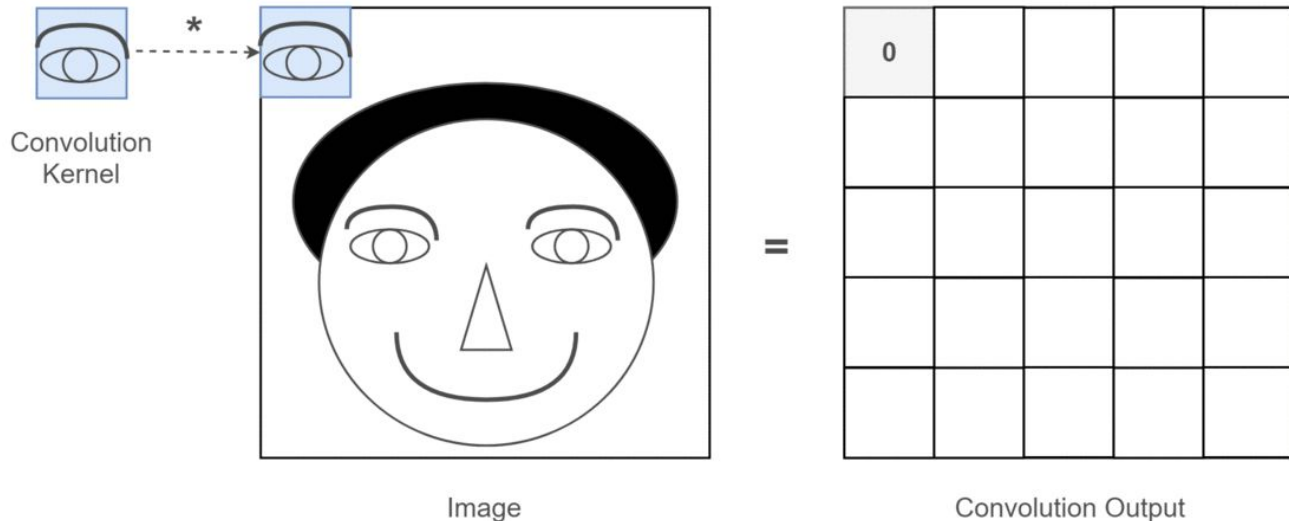
$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$



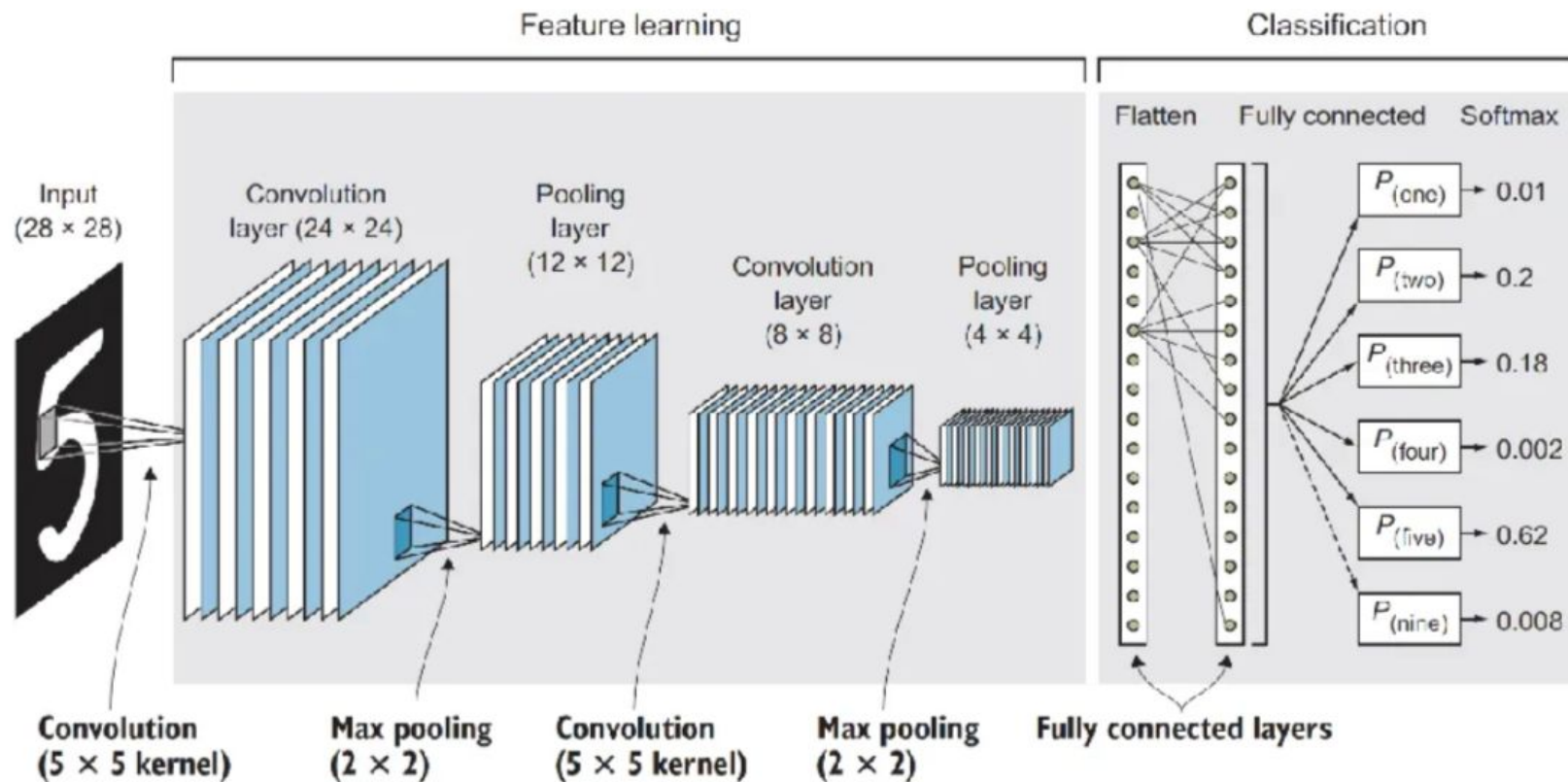
# Redes para imágenes

## Capas convolucionales



La **convolución** reduce la cantidad de parámetros que necesitamos aprender. En lugar de aprender los pesos que conectan cada píxel de entrada en una red neuronal, solo necesitamos aprender los pesos de cada filtro (Los parámetros generados por los filtros son mucho menores que los parámetros para la imagen de entrada).

Las CNN al basarse en el funcionamiento de la corteza visual también

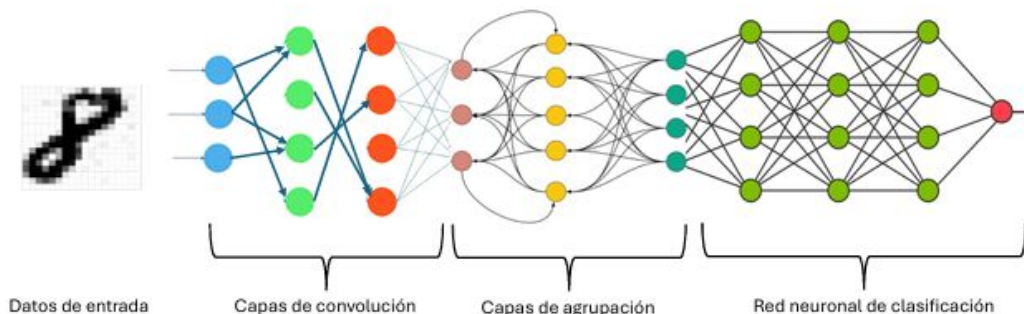


las imágenes para después poder clasificarlas. Dentro de esta capa se tienen

# Redes convolucionales

## Capas de Agrupamiento (Pooling)

El objetivo de esta capa es reducir la dimensión de los mapas de activación creados por las capas de convolución. Disminuyendo el costo computacional Los Pooling más comunes son el **max** y el **average**



El pooling ayuda de 2 maneras:

- Reduce la dimensión de entrada y conserve sólo las características importantes.
- Ayuda a identificar una imagen independientemente de su posición en una imagen.

# Redes convolucionales

## Capas de Agrupamiento (Pooling)

La segunda técnica importante sobre CNN (después de convolución) es la agrupación (pooling)

Max Pooling

|   |   |   |   |
|---|---|---|---|
| 2 | 7 | 9 | 7 |
| 4 | 9 | 1 | 4 |
| 4 | 7 | 6 | 2 |
| 5 | 1 | 8 | 2 |

|  |  |
|--|--|
|  |  |
|  |  |

Average Pooling

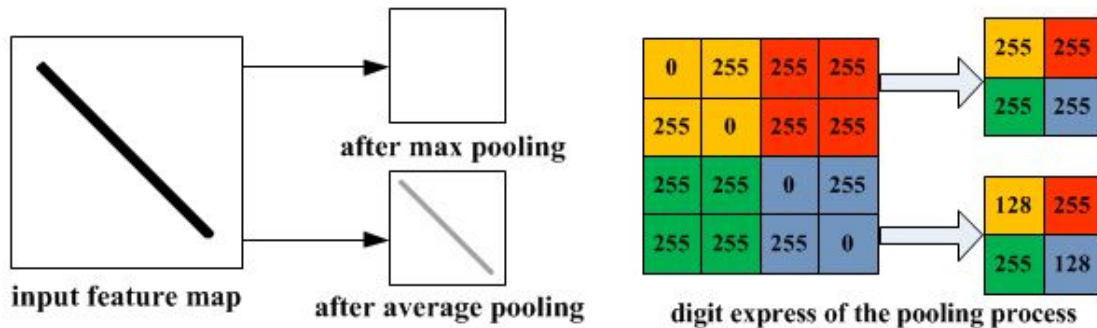
|   |   |   |   |
|---|---|---|---|
| 2 | 7 | 9 | 7 |
| 4 | 9 | 1 | 4 |
| 4 | 7 | 6 | 2 |
| 5 | 1 | 8 | 2 |

|  |  |
|--|--|
|  |  |
|  |  |

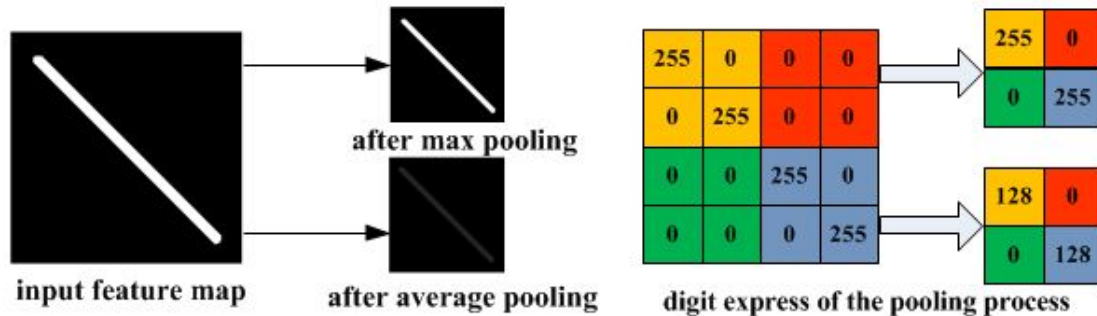


# Redes convolucionales

## Capas de Agrupamiento (Pooling)



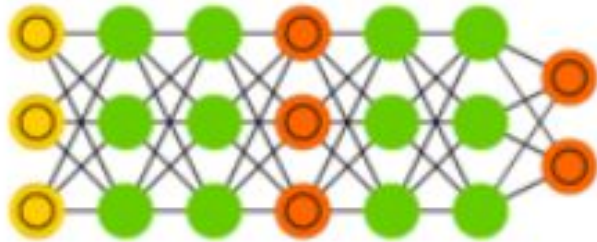
(a) Illustration of max pooling drawback



(b) Illustration of average pooling drawback

# Redes Adversarias Generativas (GAN)

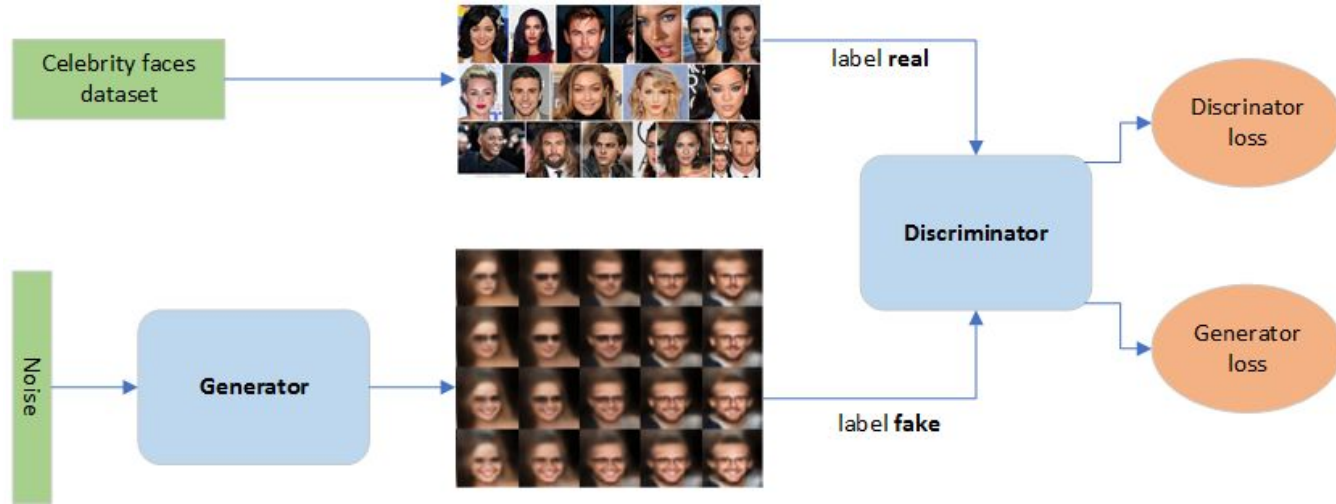
Generative Adversarial Network (GAN)



## ¿Cómo interpretarlas?

El generador crea imágenes a partir de ruido, como un pintor que inventa cuadros de la nada. El discriminador las examina y dicta si son reales; al enfrentarse una y otra vez, ambos aprenden hasta que las obras falsas resultan casi indistinguibles.

# Redes Adversarias Generativas (GAN)



La verdadera fuerza de una **GAN** surge cuando este pulso adversarial converge hacia un equilibrio

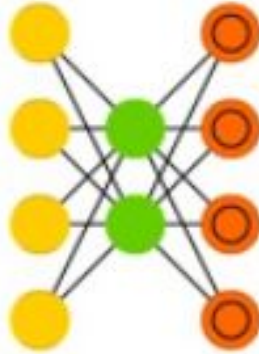
# Redes Adversarias Generativas (GAN)

- *Generar ejemplos para conjuntos de datos de imágenes*
- *Generar fotografías de rostros humanos*
- *Genera fotografías realistas*
- *Generar personajes de dibujos animados*
- *Traducción de imagen a imagen*
- *Traducción de texto a imagen*
- *Traducción semántica de imagen a foto*
- *Generación de vista frontal de cara*
- *Genera nuevas poses humanas*
- *Fotos a Emojis*
- *Edición de fotografía*
- *Envejecimiento facial*
- *Fusión de fotos*
- *Súper resolución*
- *Repintado de fotos*
- *Traducción de ropa*
- *Predicción de vídeo*
- *Generación de objetos 3D*



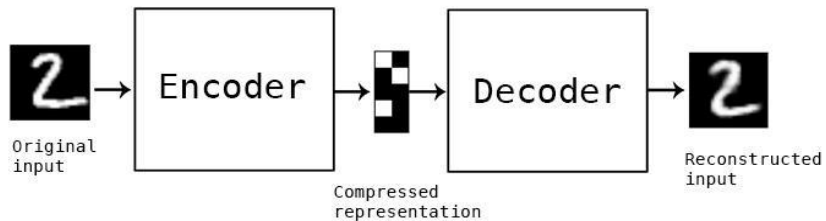
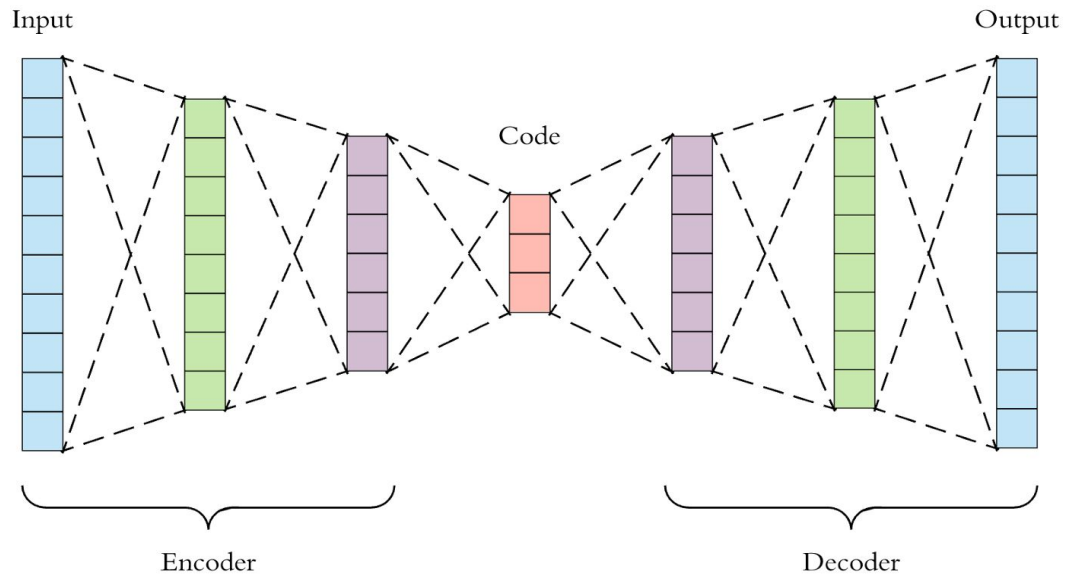
# Autoencoders

Auto Encoder (AE)

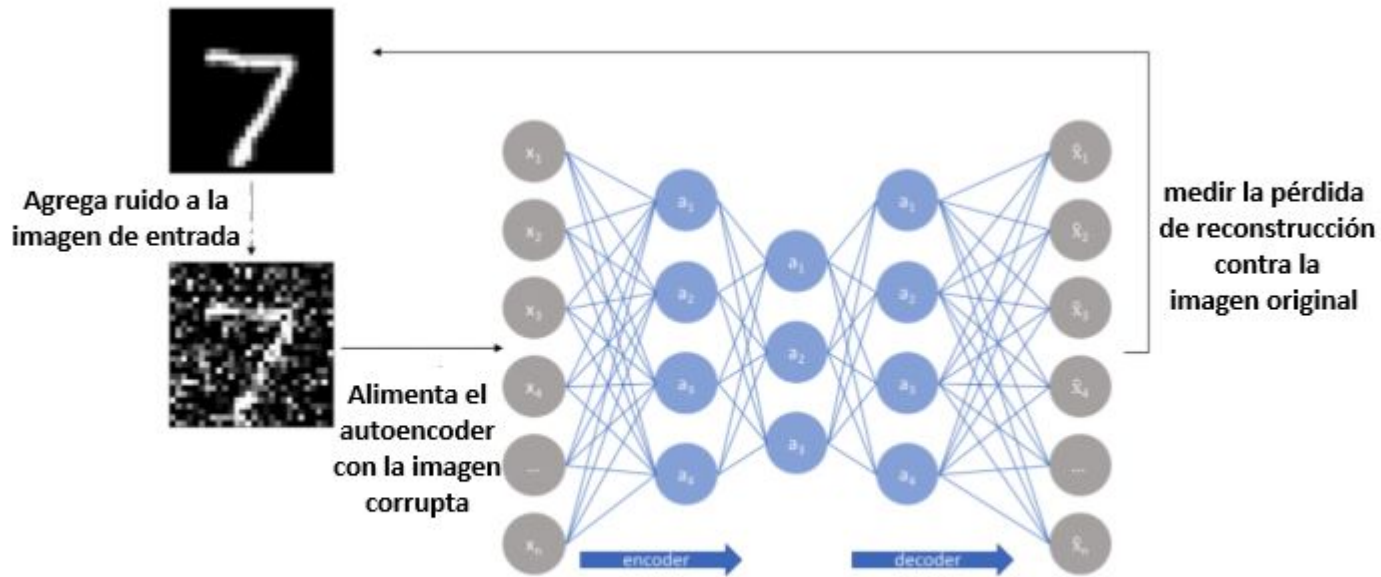


El codificador comprime la entrada en un vector latente que conserva la información clave. El decodificador lee ese vector y genera una salida coherente, ya sea traducida, reconstruida o continuada.

# Autoencoders

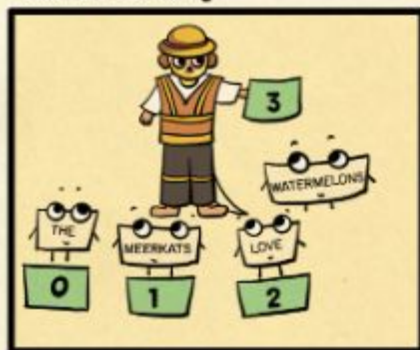


# Autoencoders





## Positional Encoding

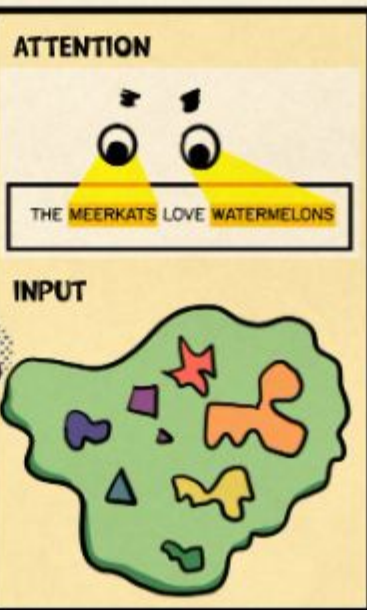


## Input

THE MEERKATS LOVE WATERMELONS.



ENCODER



# TRANSFORMER

@MIAMIAMIA0103

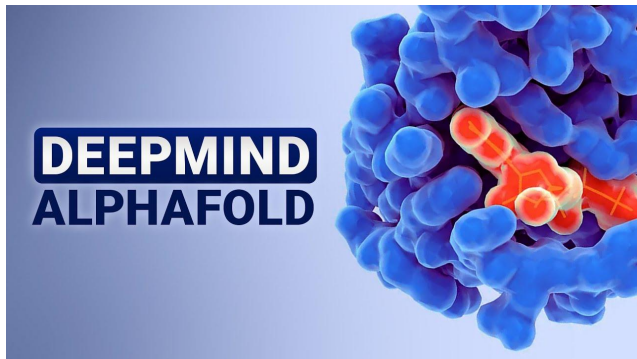
Gracias por la  
información  
relevante entendiendo  
el sentido

DECODER

SURICAT  
OUTPUT

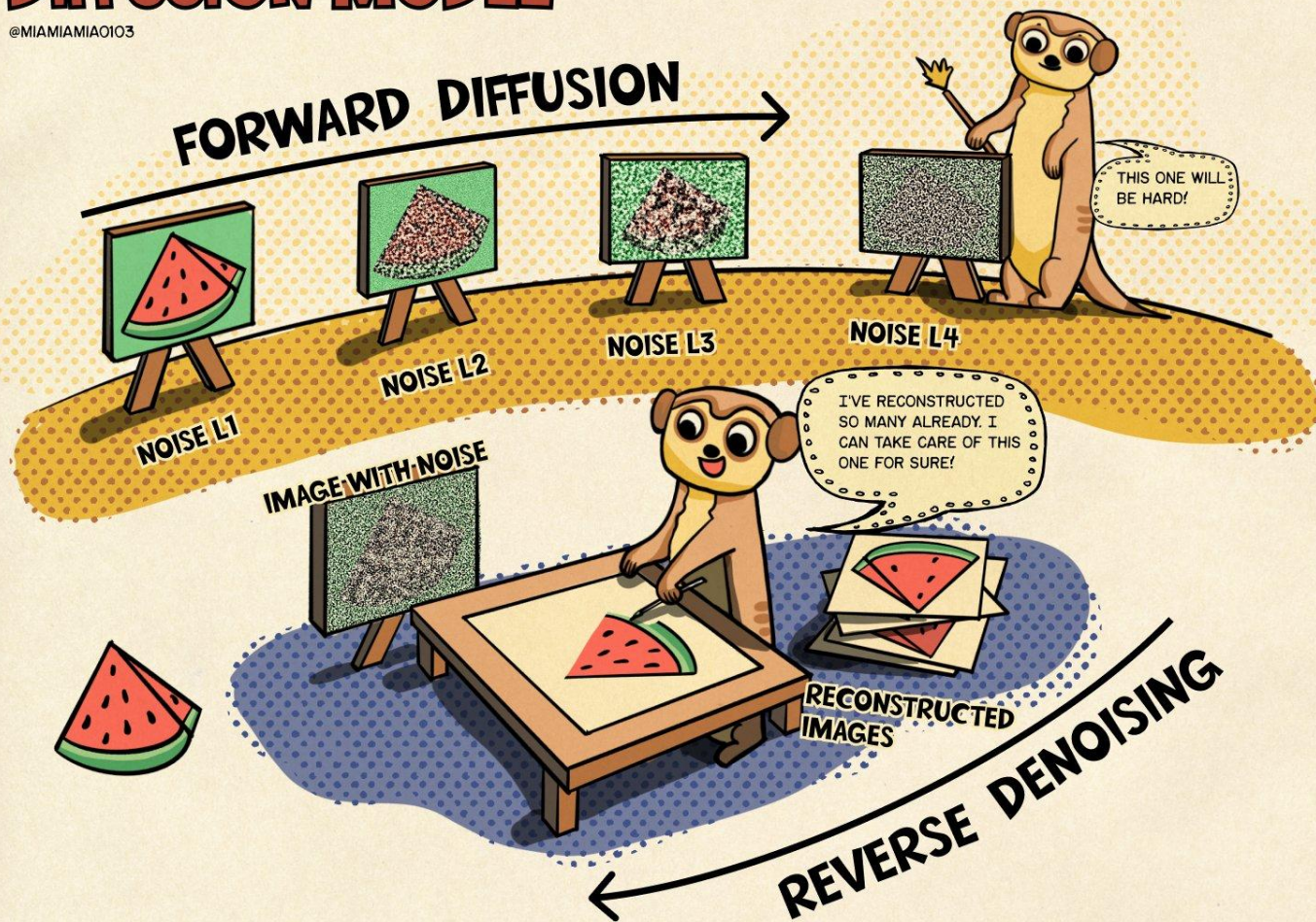


# TRANSFORMERS



# DIFFUSION MODEL

@MIAMIAMIA0103





# TRANSFORMERS

MIDJOURNEY



DALL-E 2



STABLEDIFFUSION

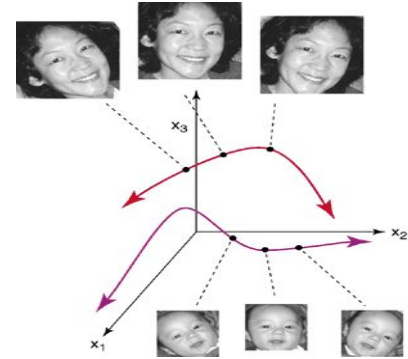


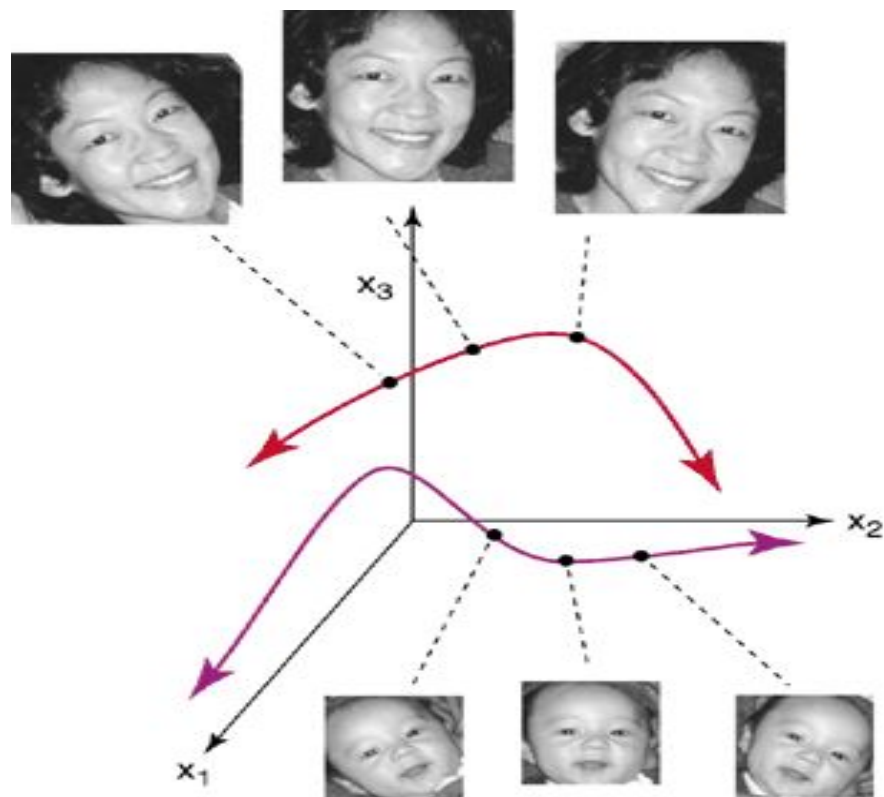
Pixar movie scene of a dark skull wizard fighting against Kermit the frog as a gladiator, incredible render, Presto

# HIPÓTESIS DE MANIFOLD

# LA HIPÓTESIS DE MANIFOLD

- Manifold es un término matemático para referirse a ciertos espacios de dimensiones arbitrarias.
- Supongamos que queremos clasificar todas las imágenes en blanco y negro de  $m \times n$  píxeles. Cada píxel tiene un valor y el conjunto de pixels va a armar una imagen con o sin sentido.
- Cada imagen puede ser pensada como un punto en un espacio de  $m \times n$  dimensiones
- Ahora pensemos solo en imágenes de Einstein. Vamos a tener restricciones en la composición de píxeles que pueden formar esas imágenes de Einstein. Una composición aleatoria de píxeles probablemente no genere una imagen de Einstein
- La hipótesis de manifold dice que este subset (de imágenes) vive en un espacio de menor dimensión, bastante más chico que  $m \times n$





# LA HIPÓTESIS DE MANIFOLD

La hipótesis de manifold explica porqué las técnicas de ML son capaces de encontrar características/variables útiles y producir predicciones precisas de datasets que tienen potencialmente dimensiones mucho más grandes (variables)

El hecho de que el dataset de interés vive en un espacio de menores dimensiones (el manifold), significa que el modelo de ML **solo** necesita focalizarse en unos pocas características claves para tomar la decisión

<https://deeptai.org/machine-learning-glossary-and-terms/manifold-hypothesis>

<https://www.youtube.com/watch?v=BePQBWPnYuE>